

TodoStock S.A.

Sistema de Gestión para Distribuidora Mayorista

Desarrollo de Sistemas Web (Back End) 2

IFTS 29 — 1° Cuatrimestre 2026

Grupo 2

Benitez Guillermo • Benitez Julian

Moreno Diego • Vigo Lucrecia

Vivar Edison Cristian

Documentación - TodoStock S.A.

Sistema de Gestión para Distribuidora Mayorista

Materia: Desarrollo de Sistemas Web (Back End) 2

Comisión: IFTS 29

Cuatrimestre: 1° Cuatrimestre 2026

Grupo: 2

Integrantes

Apellido	Nombre	Rol
Benitez	Guillermo	Tester / Documentación
Benitez	Julian	Backend Developer
Moreno	Diego	Backend Developer
Vigo	Lucrecia	Frontend / Vistas
Vivar	Edison Cristian	Arquitecto / DevOps

Repositorio

- **GitHub:** <https://github.com/jbenitez79/ifts29-backend>
 - **Video explicativo:** *[enlace al video]*
 - **Despliegue:** *[enlace a Render/Railway]*
-

1. Introducción

TodoStock S.A. es un sistema de gestión desarrollado para una empresa familiar dedicada a la comercialización mayorista de artículos de limpieza, con más de 20 años de trayectoria. La empresa atraviesa un proceso de transición generacional y presenta dificultades derivadas de una estructura jerárquica tradicional, toma de decisiones informales, y ausencia de integración entre procesos de compras, ventas y cobranzas.

El sistema permite registrar productos, proveedores, clientes, movimientos de stock y cuentas corrientes, organizando la información del proceso de compras e inventario para mejorar la trazabilidad de los productos y generar información confiable para la toma de decisiones operativas.

La aplicación está construida con **Node.js** y **Express** en el backend, **MongoDB** como base de datos, **Mongoose** como ODM, **Pug** como motor de vistas, y utiliza **JWT** (JSON Web Tokens) para autenticación y autorización por roles.

2. Objetivos

2.1 Objetivo General

Desarrollar un sistema web que permita a TodoStock S.A. gestionar eficientemente sus operaciones comerciales, integrando los procesos de compras, ventas, inventario y cobranzas en una única plataforma, facilitando la transición hacia un modelo de gestión profesional.

2.2 Objetivos Específicos

- Modelar el proceso de compras y ventas con trazabilidad completa.
 - Implementar un módulo de inventario con control de stock mínimo.
 - Representar el estado de las cuentas corrientes de clientes vinculadas a despachos y cobranzas.
 - Incorporar autenticación y autorización por roles (admin / operador).
 - Proveer una API REST y vistas HTML para la interacción con el sistema.
 - Implementar pruebas automatizadas que garanticen el correcto funcionamiento.
 - Desplegar la aplicación en la nube para acceso remoto.
-

3. Especificación de funcionalidades

3.1 Autenticación y Autorización

- **Login** con email y password, generación de JWT almacenado en cookie HttpOnly.
- **Logout** que invalida la cookie.
- **Register** de nuevos usuarios (solo admin).
- **Middleware de autenticación** que verifica el JWT en cada request a rutas protegidas.
- **Autorización por roles:** admin accede a todos los módulos; operador solo a Clientes, Pedidos y Cuentas Corrientes.

3.2 Módulo Clientes

CRUD completo con los campos: nombre, apellido, email (único), teléfono, CUIT (único), domicilio, localidad, provincia, país, código postal y fecha de nacimiento. Validaciones de campos obligatorios y unicidad.

3.3 Módulo Productos

CRUD completo con los campos: nombre, descripción, precio, stock actual y stock mínimo. Permite controlar el nivel de inventario y detectar productos por debajo del mínimo.

3.4 Módulo Proveedores

CRUD completo con los campos: nombre, CUIT (único), teléfono, email, domicilio, localidad, provincia, país, rubro, condición de pago y estado activo/inactivo.

3.5 Módulo Pedidos

CRUD completo que asocia un cliente con una lista de productos (con cantidad y precio). Calcula el total automáticamente. Estados: pendiente, aprobado, enviado, entregado, cancelado.

3.6 Módulo Cuenta Corriente

Gestiona la cuenta corriente de cada cliente con saldo, límite de crédito y estado (activo/con_deuda). Permite registrar cargos y pagos con historial de movimientos.

3.7 Dashboard

Vista principal que presenta un resumen con tarjetas de acceso rápido a cada módulo.
Adapta su contenido según el rol del usuario.

4. Requerimientos funcionales y no funcionales

4.1 Requerimientos Funcionales

Código	Descripción
RF-01	El sistema debe permitir la autenticación mediante email y password
RF-02	El sistema debe permitir el registro de nuevos usuarios (solo admin)
RF-03	El sistema debe tener roles de usuario (admin, operador)
RF-04	El sistema debe permitir CRUD de clientes
RF-05	El sistema debe permitir CRUD de productos
RF-06	El sistema debe permitir CRUD de proveedores
RF-07	El sistema debe permitir CRUD de pedidos
RF-08	El sistema debe permitir gestionar cuentas corrientes
RF-09	El sistema debe permitir registrar cargos y pagos en cuentas corrientes
RF-10	El sistema debe mostrar un dashboard con acceso a los módulos
RF-11	El sistema debe restringir productos y proveedores solo para admin
RF-12	El sistema debe mostrar stock actual de productos

4.2 Requerimientos No Funcionales

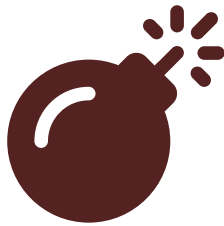
Código	Descripción
RNF-01	La aplicación debe construirse con Node.js y Express
RNF-02	Los datos deben persistirse en MongoDB usando Mongoose
RNF-03	Las contraseñas deben almacenarse hasheadas con bcrypt
RNF-04	La autenticación debe usar JWT en cookies HttpOnly
RNF-05	Las vistas deben renderizarse con Pug
RNF-06	La API debe responder en formato JSON
RNF-07	El sistema debe incluir middleware de manejo de errores
RNF-08	El código debe seguir el patrón MVC
RNF-09	Debe incluir pruebas automatizadas con Jest + Supertest

Código	Descripción
RNF-10	Las variables de entorno deben estar separadas en .env

5. Diagramas UML

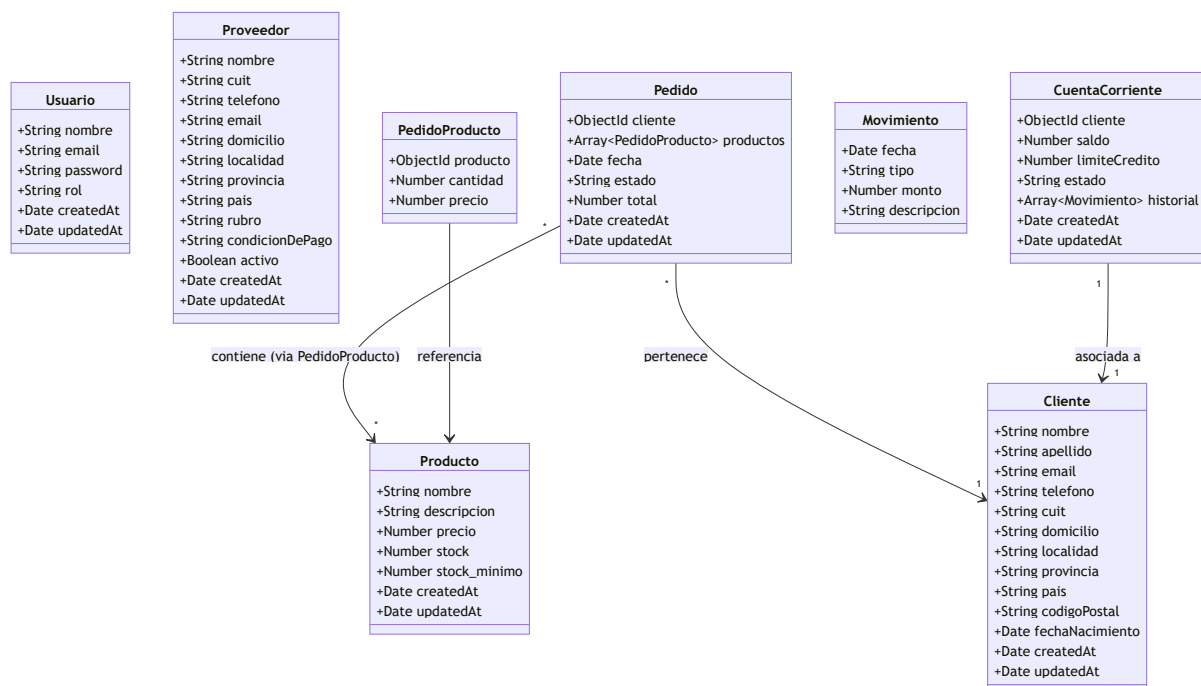
Los siguientes diagramas fueron generados con Mermaid y describen la arquitectura del sistema.

1. Diagrama de Casos de Uso

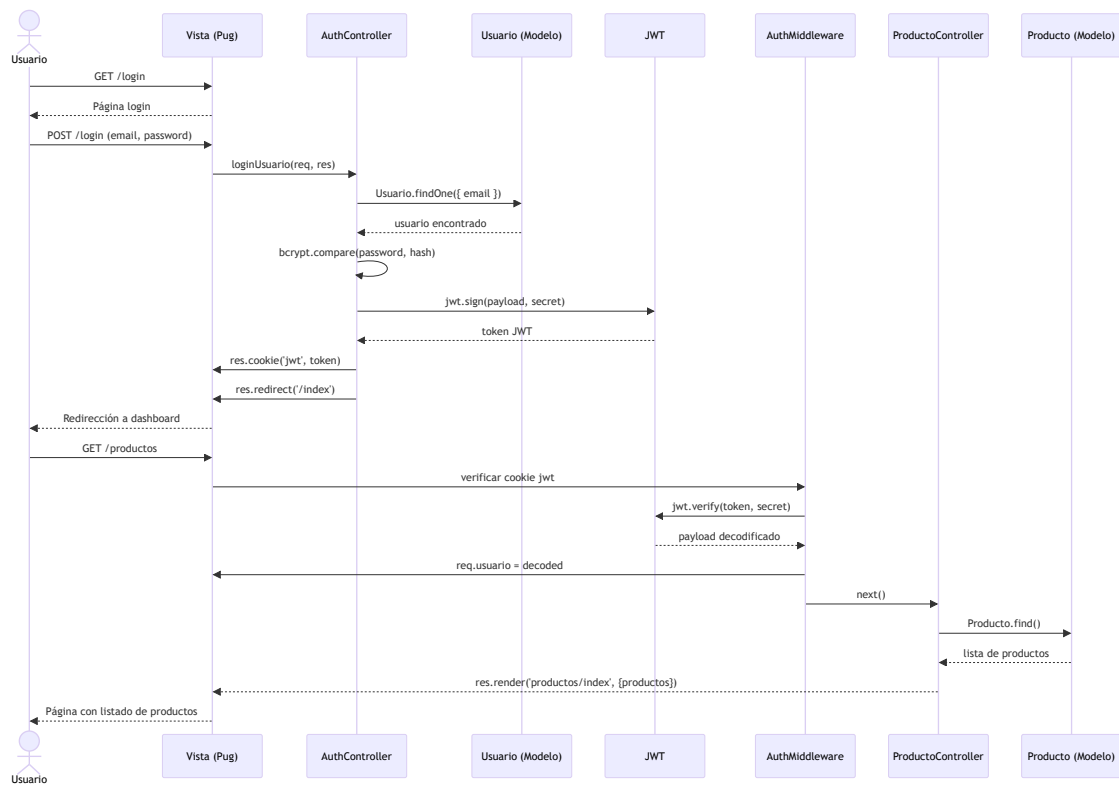


Syntax error in text
mermaid version 10.9.6

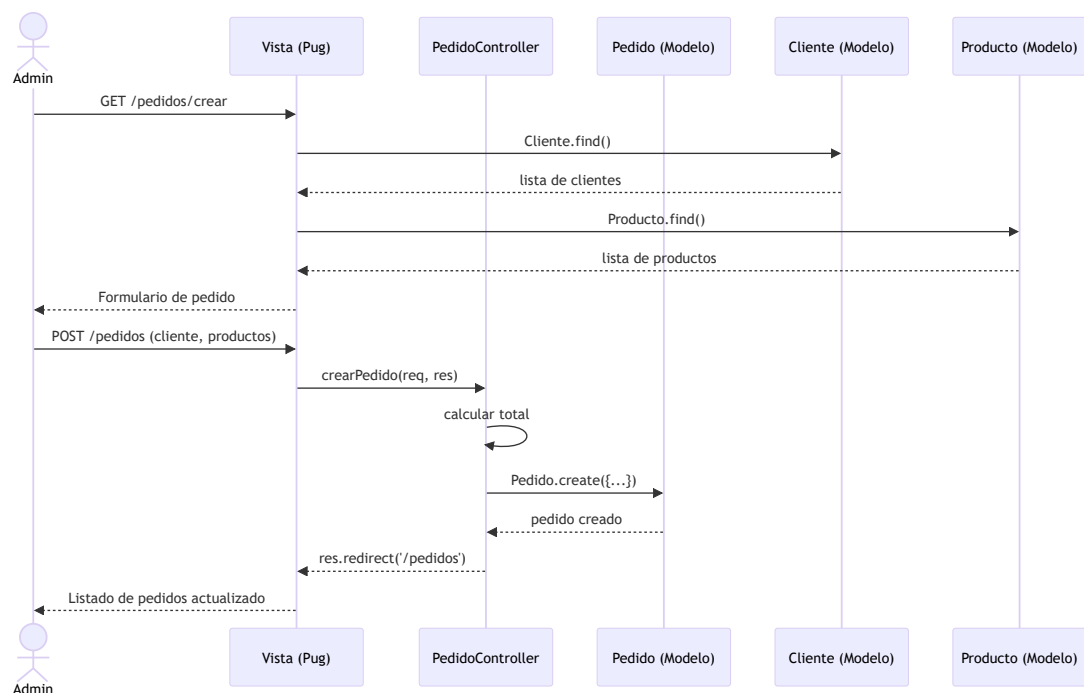
2. Diagrama de Clases



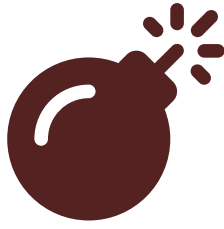
3. Diagrama de Secuencia - Login y Consulta de Productos



4. Diagrama de Secuencia - Creación de Pedido

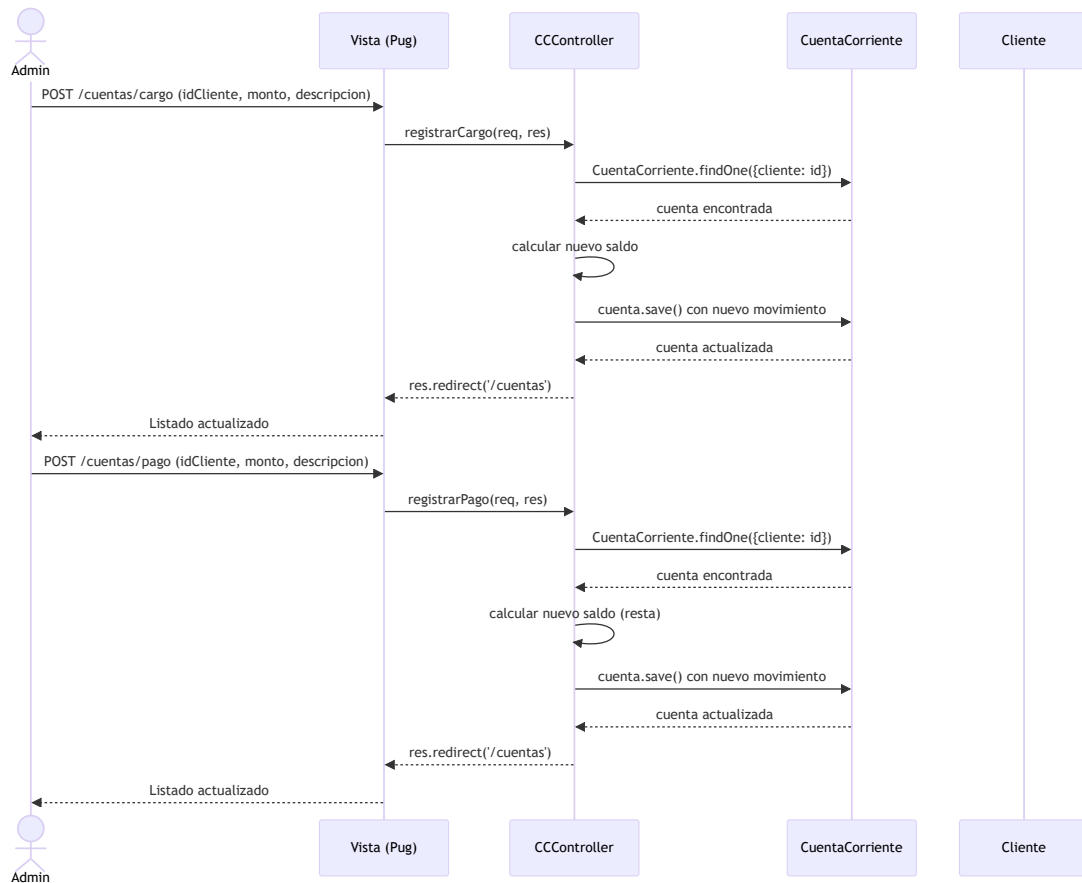


5. Modelo Entidad-Relación (DER)



Syntax error in text mermaid version 10.9.6

6. Diagrama de Secuencia - Cargo/Pago en Cuenta Corriente



Los diagramas completos se encuentran en el archivo [diagramas.md](#) de esta misma carpeta.

5.1 Diagrama de Casos de Uso

El sistema cuenta con dos actores:

- **Administrador:** Accede a todos los módulos (Clientes, Productos, Proveedores, Pedidos, Cuentas Corrientes, Usuarios, Dashboard).
- **Operador:** Accede solo a Clientes, Pedidos, Cuentas Corrientes y Dashboard.

Ambos actores deben autenticarse para acceder al sistema.

5.2 Diagrama de Clases

Se identifican las siguientes entidades principales:

- **Usuario:** nombre, email, password, rol
- **Cliente:** datos personales y de contacto (referenciado por Pedido y CuentaCorriente)
- **Producto:** nombre, descripción, precio, stock, stock_minimo
- **Proveedor:** datos comerciales y de contacto (entidad independiente)
- **Pedido:** cliente, productos (embedded), fecha, estado, total
- **CuentaCorriente:** cliente, saldo, límite, historial de movimientos (embedded)

5.3 Diagramas de Secuencia

Se modelaron dos flujos principales:

1. **Login + consulta de productos:** Muestra el flujo completo desde el login, generación del JWT, verificación del token por el middleware, y consulta a la base de datos.
2. **Creación de pedido:** Muestra la interacción entre vistas, controlador y modelos para crear un pedido con productos seleccionados.
3. **Cargo/Pago en Cuenta Corriente:** Muestra el registro de movimientos financieros con actualización de saldo.

5.4 Modelo Entidad-Relación

Las relaciones principales son:

- **Cliente** 1 → N **Pedido** (un cliente puede tener varios pedidos)
 - **Pedido** N → M **Producto** (a través de PedidoProducto embedded)
 - **Cliente** 1 → 1 **CuentaCorriente** (cada cliente tiene una cuenta)
 - **CuentaCorriente** 1 → N **Movimiento** (embedded, historial de cargos/pagos)
-

6. Explicación del funcionamiento

6.1 Arquitectura

El sistema sigue el patrón **MVC (Model-View-Controller)**:

- **Modelos:** Definen los esquemas de datos con Mongoose, incluyendo validaciones, relaciones (refs) y valores por defecto.
- **Vistas:** Templates Pug que renderizan HTML del lado del servidor. Separadas por módulo en carpetas dentro de `views/`.
- **Controladores:** Lógica de negocio que procesa requests y responde con JSON (API) o renderiza vistas.
- **Rutas:** Definen los endpoints y aplican middleware de autenticación y autorización.

6.2 Flujo de Autenticación

1. El usuario ingresa email y password en el formulario de login.
2. El controlador busca el usuario en MongoDB y compara la password con bcrypt.
3. Si es válido, genera un JWT con el payload {id, nombre, rol, email} y lo almacena en una cookie HttpOnly.
4. Cada request a rutas protegidas pasa por el middleware `authMiddleware` que verifica el JWT.
5. Si el token es válido, inyecta `req.usuario` y `res.locals.usuario` para uso en controladores y vistas.

6.3 Flujo de Autorización

- El middleware `authorizeRoles()` recibe los roles permitidos como argumentos.
- Se usa como segundo middleware después de `authMiddleware`.
- Productos y Proveedores están restringidos a `admin`; Clientes, Pedidos y Cuentas son accesibles por ambos roles.

6.4 API REST vs Vistas

Cada módulo expone dos interfaces:

- **API REST:** Respuestas en JSON, útiles para integración con frontends o pruebas automatizadas.
- **Vistas HTML:** Renderizadas con Pug para uso directo por usuarios desde el navegador.

Ambas interfaces comparten los mismos controladores; la diferenciación se hace por el tipo de request (Accept header implícito por ruta).

6.5 Manejo de errores

El middleware global `errorHandler` captura:

- **ValidationError de Mongoose:** Campos requeridos, tipos incorrectos, etc.
 - **Errores de duplicado (código 11000 de MongoDB):** Email o CUIT duplicados.
 - **Errores genéricos:** Respuesta 500 con detalle del error en entorno de desarrollo.
-

7. Mejoras y correcciones realizadas

7.1 Correcciones de la Versión 1.0 (TP2)

Problema	Solución
Sintaxis ternaria partida en Pug	Reescritura de condicionales en <code>clientes/detalle.pug</code>
<code>fechaNacimiento</code> sin null check	Validación previa antes de formatear
<code>returnDocument</code> inconsistente vs <code>new:true</code>	Unificación a <code>new:true</code> en todos los updates
Footer contenido "con JSON" legacy	Reemplazo por texto profesional
Dependencias no utilizadas (~70)	Limpieza a ~8 dependencias esenciales

7.2 Mejoras implementadas (Versión 1.1)

Mejora	Descripción
Autenticación JWT	Login/register/logout con tokens y cookies HttpOnly
Autorización por roles	Middleware <code>authorizeRoles()</code> con roles admin/operador
Middleware de errores	Handler global con manejo de errores Mongoose y MongoDB
Pruebas automatizadas	44 tests con Jest + Supertest (CRUD + autenticación)
Seed de datos	Script para generar datos de prueba
Documentación	README, guías de ejecución local y pruebas, diagramas
Variables de entorno	<code>.env.example</code> con configuración separada
Dashboard adaptativo	Ocultamiento de opciones según el rol
Botón de cerrar sesión	Logout funcional en el navbar
Arquitectura MVC	Separación clara en modelos, vistas, controladores y rutas

8. Documentación de pruebas

8.1 Estrategia

Se implementaron pruebas automatizadas con **Jest** y **Supertest** que verifican:

- **Integración:** Todos los endpoints responden correctamente (GET).
- **CRUD completo:** Creación, lectura, actualización y eliminación de cada entidad.
- **Autenticación:** Login exitoso, login fallido, logout, registro, duplicados, roles inválidos.
- **Autorización:** Middleware authorizeRoles bloquea operadores en rutas de admin.
- **Cuenta Corriente:** Verificación de saldo después de cargos y pagos.

8.2 Suites de pruebas

Suite	Tests	Objetivo
<code>allEndpoints.test.js</code>	5	Verificar que todos los GET devuelvan 200
<code>cliente.qa.test.js</code>	5	CRUD completo de clientes
<code>producto.qa.test.js</code>	5	CRUD completo de productos
<code>proveedor.qa.test.js</code>	5	CRUD completo de proveedores
<code>pedido.qa.test.js</code>	5	CRUD completo de pedidos
<code>cuentaCorriente.qa.test.js</code>	6	CRUD + cargo/pago con verificación de saldo
<code>authorize.test.js</code>	5	Middleware authorizeRoles (admin pasa, operador 403, sin auth 401)
<code>auth.test.js</code>	8	Login (válido/inválido), logout, register (éxito/duplicado/rol inválido)

8.3 Resultados

Total: 8 suites, 44 tests, 0 fallas

Todos los tests se ejecutan con `npm test` (configurado con `cross-env NODE_ENV=test`) y utilizan la base de datos MongoDB local.

8.4 Procedimiento de ejecución

```
npm test
```

Requisitos:

- MongoDB en ejecución (localhost:27017)
 - Dependencias instaladas (`npm install`)
-

9. Asignación de roles y responsabilidades

Integrante	Tareas asignadas
Benitez Guillermo	Diseño y ejecución de pruebas automatizadas, documentación de pruebas, control de calidad
Benitez Julian	Implementación de CRUDs (Productos, Proveedores), rutas, validaciones del lado del servidor
Moreno Diego	Implementación de CRUDs (Clientes, Pedidos, Cuenta Corriente), lógica de negocio, relaciones entre entidades
Vigo Lucrecia	Diseño de vistas Pug, maquetado de formularios y listados, estilos, experiencia de usuario
Vivar Edison Cristian	Arquitectura del sistema, configuración de base de datos, autenticación JWT, middlawares, pruebas, documentación, despliegue

10. Uso de IA

Durante el desarrollo de este proyecto se utilizó **opencode** (asistente de IA para terminal) como herramienta de apoyo para:

- Generación de código repetitivo (CRUDs, rutas, tests).
- Corrección de errores de sintaxis (Pug, JavaScript).
- Sugerencias de estructura y buenas prácticas.
- Generación de documentación y diagramas.

Declaración: Todo el contenido generado con IA fue revisado, adaptado y verificado por el equipo. El código generado fue probado con la suite de tests automatizados antes de integrarse. Las decisiones arquitectónicas y de diseño fueron tomadas por el equipo.

11. Conclusión

¿Qué aprendimos este cuatrimestre?

Aprendimos a desarrollar una aplicación web completa con Node.js y Express, integrando base de datos MongoDB, autenticación JWT, autorización por roles, manejo de errores, y pruebas automatizadas. Comprendimos la importancia de la arquitectura MVC y las buenas prácticas de desarrollo backend.

¿Qué dificultades tuvimos y cómo las resolvimos?

- **Migración de JSON a MongoDB:** Requirió reestructurar todo el código para usar Mongoose. Se resolvió gradualmente, migrando un módulo a la vez.
- **Autenticación con JWT:** La implementación de cookies HttpOnly y middleware de autorización presentó desafíos de integración con las vistas Pug. Se resolvió con el uso de `res.locals` para disponibilizar el usuario en todas las vistas.
- **Pruebas con autenticación:** El bypass de auth en modo test facilitó las pruebas de CRUD pero requirió tests específicos para la lógica de autenticación.
- **Express v5 vs v4:** Incompatibilidades con method-override requirieron ajustes en la configuración.

¿Qué parte del desarrollo Back End nos interesó más?

La implementación de la autenticación y autorización con JWT resultó particularmente interesante, así como el diseño de las relaciones entre modelos con Mongoose (refs, populate, embedded documents).

¿Qué deberíamos reforzar?

- Operaciones atómicas con MongoDB (\$inc para stock y saldos).
- Manejo de sesiones y refresco de tokens JWT.
- Documentación automatizada de APIs (Swagger/OpenAPI).
- Prácticas de seguridad adicionales (helmet, rate limiting, CORS).

12. Bibliografía

- **Node.js:** <https://nodejs.org/docs/>
 - **Express.js:** <https://expressjs.com/en/guide/routing.html>
 - **MongoDB:** <https://www.mongodb.com/docs/>
 - **Mongoose:** <https://mongoosejs.com/docs/>
 - **Pug:** <https://pugjs.org/>
 - **JWT (jsonwebtoken):** <https://github.com/auth0/node-jwebtoken>
 - **bcrypt:** <https://github.com/kelektiv/node.bcrypt.js>
 - **Jest:** <https://jestjs.io/docs/>
 - **Supertest:** <https://github.com/ladjs/supertest>
 - **Modelo de documentación DSWB:**
<https://docs.google.com/document/d/14k9j3AlcBbHFPW5LdOXOLF-IC8oC-r05mFa9-jwLBgw>
 - **Bibliografía DSWB:**
https://docs.google.com/document/d/1A11xLgq5m_24dQjyy4h73Dyc7t0ynVqGU3jlaRYn6G8
 - **OpenCode (IA):** <https://opencode.ai>
-

Anexos

A. Enlaces

- **Repositorio GitHub:** <https://github.com/jbenitez79/ifts29-backend>
- **Video explicativo:** *[enlace al video grupal]*
- **Despliegue en producción:** *[enlace a Render/Railway]*

B. Comandos útiles

```
# Instalar dependencias
npm install

# Ejecutar en desarrollo
npm run dev

# Ejecutar en producción
npm start

# Ejecutar pruebas
npm test

# Generar datos de prueba
npm run seed
```